PROJECT REPORT

AI CYBER THREAT DETECTION PLATFORM

System Analysis & Project Summary

STUDENT	Mohammed daiyan
USN	4ni24cs416
SEMESTER	7th

A full-stack Security Operations Center (SOC) prototype for network telemetry analysis, machine-learning threat classification, and incident investigation.

MAJOR PROJECT | COMPUTER SCIENCE & ENGINEERING

01 | Project Overview

Purpose and scope

This repository implements a Security Operations Center console designed to turn network traffic into actionable threat alerts. It combines flow preparation and packet-capture ingestion, supervised classification, a persistent incident store, and an interactive analyst dashboard. The workflow supports triage from detection through evidence review and reporting.

The system is an academic demonstrator and prototype. Its end-to-end breadth is useful for explaining how telemetry, machine learning, and analyst response can fit together; production readiness requires independent model validation, hardened access controls, and careful operational safeguards.

System at a glance

User interface	React 19 and Vite; dashboard pages for overview, alerts, analytics, model comparison, and settings.
API and storage	FastAPI REST and WebSocket service with SQLAlchemy; SQLite default and configurable PostgreSQL.
Detection	Serialized Random Forest classifier and checked-in comparison metrics for four candidate models.
Inputs	Prepared flow dataset, uploaded PCAP, optional live interface capture, and traffic simulation.
Analyst outputs	Live alerts, feature explanations, MITRE ATT&CK context, PDF reports, and firewall rule exports.

Repository map

backend/app hosts the API, configuration, persistence models, schemas, reporting, explainability, and optional AI service. **src** contains data preparation, training, inference, packet parsing, live sniffing, and traffic simulation. **ml** contains the model artifact and metrics; **data** contains telemetry and capture files; **frontend/src** holds pages and reusable SOC components.

02 | Architecture & Data Flow

The design separates ingestion, intelligence, persistence, and presentation. Traffic input is transformed into flow features; the model classifies each flow; the backend enriches and persists the event; then the dashboard receives records through API queries or a WebSocket push stream.

Operational flow

1 Ingest	Load prepared records, parse a PCAP, or capture traffic through the optional sniffer.
2 Extract	Aggregate packets into flow-level records and align the feature vector to the inference schema.
3 Classify	Apply the saved classifier and derive a threat label and severity/context.
4 Persist & stream	Store threat records through SQLAlchemy and notify connected dashboards.
5 Investigate	Review event details, feature attribution, ATT&CK mapping, and supporting telemetry.
6 Respond	Create a formal report or export an operator-reviewed firewall/IDS rule.

Technology and interfaces

The backend is Python-based, using FastAPI, Pydantic, SQLAlchemy, pandas, NumPy, and scikit-learn. The frontend uses React, React Router, Axios, Recharts, and Lucide icons. REST endpoints support dashboard operations; WebSockets are intended to deliver new alerts without page refresh. The database configuration supports local SQLite and a PostgreSQL connection string.

Optional integrations

SHAP is used to attribute tree-model decisions in the investigation experience. A Gemini service can generate explanatory narrative when configured. ReportLab generates incident PDFs. These enrichments should remain traceable to source event data, and AI-generated narrative should be reviewed by an analyst.

Important integration boundary

Training data and live PCAP parsing must produce compatible features with matching names, units, ordering, and preprocessing. This feature parity is central to meaningful predictions; a mismatch can make offline benchmark performance irrelevant to live traffic.

03 | Machine Learning & Analyst Features

Data and model

The repository includes a data pipeline, a multi-model trainer, inference code, a serialized Random Forest, and model metadata. The current README and metrics describe six classes: Benign, Botnet, Brute Force, DDoS, PortScan, and Web Attack. The metrics file reports Random Forest accuracy of 99.50% and macro F1 of 99.37% over a stated 2,400-sample test set. It reports Hist Gradient Boosting at 99.50% accuracy, MLP at 99.00%, and Logistic Regression at 98.38%; Random Forest is selected as active.

Interpretation of reported scores

These values are claims recorded in the repository and were not independently reproduced for this report. Supporting project documents refer to NSL-KDD and synthesized records, while the README/metrics refer to CIC-IDS2017. The dataset description and provenance therefore need reconciliation. High scores can also be affected by leakage, duplicate or related samples, class balance, and split design.

Analyst-facing capabilities

- **Alert investigation:** inspect classification and flow fields alongside geographic and severity context.
- **Explainability:** use SHAP feature attribution to show signals that pushed a tree-model prediction.
- **Threat context:** associate incident types with MITRE ATT&CK techniques.
- **Live awareness:** dashboard updates and animated threat-map visualization.
- **Incident artifacts:** generate PDF reports and export Windows, Linux, Suricata, or Snort rules.

Validation plan

Document exact dataset versions and licenses; create leakage-safe, reproducible partitions; publish per-class precision/recall/F1, confusion matrices, and calibration; validate on independently captured traffic; and measure end-to-end throughput and latency. Version the model and feature schema with each stored prediction.

04 | Readiness & Recommendations

Strengths

The project connects many SOC concepts in one demonstrable system: multiple traffic inputs, model comparison, explainability, live event delivery, analyst views, and response artifacts. The codebase has recognizable module boundaries, local sample data, a saved model, dependency manifests, and setup documentation.

Priority improvements

- **Data integrity:** reconcile dataset references and distinguish original benchmark data from synthetic or transformed samples.
- **Secure ingestion:** enforce upload and packet limits, validate malformed files, apply timeouts, and isolate untrusted parsing.
- **Service security:** add authentication and authorization, protect secrets, review network exposure/CORS, apply rate limits, and audit access to incidents.
- **Safe containment:** separate rule generation from execution; require least privilege, explicit approval, audit records, validation, and rollback for host firewall changes.
- **Model operations:** monitor drift and false positives, version models and feature schemas, and provide analyst feedback workflows.
- **Reliability:** verify API contracts, parser edge cases, model loading, WebSocket reconnects, report generation, and database lifecycle.

Conclusion

The platform is a broad academic SOC prototype that demonstrates how telemetry ingestion, supervised detection, explainable investigation, live visualization, and incident reporting can work together. The clearest path to a credible operational system is to establish reproducible dataset and model evidence, verify parity between offline and live features, and harden security-sensitive ingestion and containment paths.

Report basis: repository README, implementation plan, walkthrough, source layout, dependency manifests, and checked-in model metrics. Benchmark figures are reported as stored, not independently verified.